

Improving Mechanical Performance for Extreme Environments

Project Description

WSU Mechanics and Materials Laboratory



WSUM-Python Apps

Nelson Goosman

Kenan Anderson

Nicholas Lopez

2.13.2025

Table of Contents

I.	Introduction	3
II.	System Overview	3
III.	Architecture Design	3
	III.1 Overview	3
	III.2 Subsystem Decomposition	4
	III.2.1 Interface/Loading Subsystem	5
	III.2.2 Batch Segmentation Subsystem	6
	III.2.3 Data Visualization/Analysis Subsystem	6
IV.	Data Design	7
V.	User Interface Design	7
VI.	Glossary	8
VII.	References	8
VIII.	Appendices	9

I. Introduction

The Multiscale Mechanics and Materials Laboratory at Washington State University's School of Mechanical and Materials Engineering is developing an application for an automated analysis of microscopic images from high-velocity material impact tests. These tests, conducted on materials such as boron carbide ceramics and niobium-titanium alloys, help researchers understand structural failure under extreme conditions found in aerospace, defense, and space travel applications. Dr. Zare's laboratory combines data science and material research to analyze the mechanical performance of these materials.

This project aims to enhance an existing image analysis program, expanding its capabilities to improve accuracy, efficiency, and automation. The program will integrate advanced image segmentation techniques, batch processing for multi-image analysis, and statistical methods to extract key material failure metrics. Additionally, machine learning will be incorporated to predict material properties based on extracted features and impact parameters, ultimately assisting in the design of more resilient materials.

By optimizing the pipeline of data collection, image segmentation, and predictive analysis, this project aims to provide a powerful and generalized framework for materials research, enabling precise assessments of impact resistance and structural integrity.

II. System Overview

The functionality of the application right now is automated segmentation and analysis of images from an electronic microscope. There are currently three main points to our application, image segmentation, statistical analysis, and an interface where the user can interact with the various components. Currently for the design we are using python and various libraries for segmentation, analysis, data, and the graphical user interface. For the current project lifecycle and design, we are currently using the SCRUM sprint model for our sprints and various changes we are making to improve the application. In the future, we are considering a design that is scalable to a wide range of needs, creating easy interaction from a user standpoint and modularity when it comes to different needs in segmentation and statistical analysis.

III. Architecture Design

III.1 Overview

Our project team is continuing with the existing application's Call and Return architecture substyle - the Object Oriented Architectural Model. Given Python's exclusive usage in the project (with the addition of MATLAB), we agree that the Object Oriented model best conforms with Python in prioritizing scalability and simplicity in documentation. 'Diagram 1' in the Appendix below is a current reflection of the overarching project architecture in UML block diagram format.

The Diagram starts with the executable/program launch. This talks directly to the main.py file which is the general project wrapper. This talks to the GUI handler, which will create however many required ImageHandler objects that have their own parameter popups. The GUI handler also talks to the segmentation handler for general segmentation parameters. It then moves to the Image Segmentation script for performing the segmentation on the image, given the set parameters. Once segmented, the analysis script(s) will create the resultant image/pdfs that include Histograms, marked images, and plotted graphs. These are displayed in the GUI for optional, additional analysis as well as being outputs in the designated output folder structure.

III.2 Subsystem Decomposition

Our project team has analyzed and decomposed the existing segmentation pipeline to create isolated subsystems that perform critical functionality in the application. These subsystems were formed with the intent to best distribute decoupled blocks for individual developers to enhance and develop. The subsystems include User, Interface/Image Loading, Image/Batch Segmentation, and Data Visualization/Analysis. These subsystems succinctly encompass all blocks/processes described in 'Diagram 1'. In the appendix below, 'Diagram 2' is a simplified and refactored diagram that reflects these subsystems with required dependencies, provided services, and wrapped architecture blocks.

Similar to 'Diagram 1', the new diagram starts with a User that extends the executable/GUI. The User both services and depends on Interface/Image Loading as a system wrapper. Interface/Image Loading depends on the User input and potentially Data Visualization/Analysis output. It services the User and the Image/Batch Segmentation as it extends the project wrapper/GUI, Image Handler, corresponding Popup Dialogues, and File Outputs. The Image/Batch Segmentation takes the output from the Interface/Image Loading to segment the images using the Segmentation Handler and Image Segmentation components. It outputs the resultant segmentation to Data Visualization/Analysis to extend Image Analysis, before cyclically moving its output back to the Interface/Image Loading for output or re-analysis.

III.2.1 Interface/Loading Subsystem

III.2.1.1 Description

The Interface/Loading subsystem holds responsibility over the GUI and how the User uploads images/inputs data. This is the only subsystem the User will have access to, as this provides a layer of abstraction between the outside world and the backend program implementation. The GUI itself is straightforward; providing a way for the user to upload images, input parameters, and see the visualized data. Of equal importance, this subsystem takes the images that have been uploaded and creates an ImageWidget object. The ImageWidget object is a custom class that contains the image and image information to be processed later in the program. There is also a list of default parameters that can be modified for different segmentation algorithms or visualized outputs. The ImageWidget(s) and list(s) of parameters are passed on to the Batch Segmentation subsystem to be segmented before ultimately being processed and analyzed.

III.2.2.2 Concepts and Algorithms Generated

There are no explicit algorithms for the GUI or loading of images. However, our team has decided on specific concepts/solutions to refactoring the loading of images. As we ultimately want to be able to process multiple images for batch segmentation, we have to extend the existing program from a single image upload to a multi-image pipeline. While there exists the single image implementation of static, global variables for both parameters and image metadata, the new plan involves creating a specialized class that wraps this existing functionality to ultimately create a list of however many images the User may choose to upload. This simplifies the segmentation and analysis pipeline, while enabling scalability for the future. For the GUI itself, we intend to shift from a code-based PyQt5 implementation to a PyQt5 Designer structure, with a '.ui' file for easier UI modification and scalability.

Services Provided		
Service Name	Provided To	Description
Parameter Gathering	Batch Segmentation	Gathers all the possible image and segmentation parameters desired by the User to be passed along to the segmentation and analysis algorithms
Image Gathering/Transformation	Batch Segmentation	Similarly gathers all the desired images and transforms them into our ImageWidget class to be passed along to the segmentation and analysis algorithms
Intuitive Interface	User	An accessible, intuitive way for a generalized User to use the full extent of our application

Services Required	
Service Name	Provided From
User Input	User
Processed Images/Plots/Histograms	Data Visualization/Analysis

III.2.2 Batch Segmentation Subsystem

III.2.2.1 Description

This subsystem is responsible for the image processing handled by this application. It is designed to iterate over each image loaded in by the user and run the selected algorithm on each one. It provides both the visualization as well as the segmentation algorithms. This data is then passed off to other subsystems (interface and data visualization) to display to the user. The raw data generated by this component is also available for the user to download.

III.2.2.2 Concepts and Algorithms Generated

The previous capstone group left off on two main segmentation algorithms: adaptive thresholding and the watershed method. Their methodology was to apply the watershed algorithm to the pores in the SEM images, which finds “peaks” and “valleys” in the image. To look at the imperfections in an image, the application runs adaptive thresholding, which works better than the watershed method to segment individual imperfections. This allows for better visualization of the final image. We are taking over where they left off to refine these algorithms to apply them to a set of images rather than just one. This allows the user to generate more data to gain greater insight on a given material. Additionally, we are looking into a deep learning approach called U-Net. This is a single shot pretrained model created to segment biomedical imagery, but has won awards for its performance on SEM image datasets. This will give the researcher a greater range of tools to use, allowing them to find the segmentation algorithm that works best on their dataset.

Services Provided		
Service Name	Provided To	Description
Data Gathering	Data Analysis Component	Runs the segmentation analysis on a dataset, then merges all relevant data generated on each image into one. This can then be exported to the user or used to display different figures for statistical analysis.
Segmentation Image Generation	Interface	This generates a visualization of the selected image and passes it off to the interface to display. It also downloads the image to the users computer.

Services Required	
Service Name	Provided From

User Input	User
Image Selections	Interface

III.2.3 Data Visualization/Analysis Subsystem

III.2.3.1 Description

This subsystem is responsible for direct user interaction with visualized data, given from any form of segmentation. Once an image has gone through segmentation, it is passed onto another file, where it looks at the impurity data which is then used to create the statistical distributions that the user can see. Within all the data visualization, the user can see and interact with a segmented image, histograms, and graphs showing the various statistical distributions given from the impurity data.

III.2.3.2 Concepts and Algorithms Generated

With the data visualization and analysis subsystem, the previous capstone group left off with a variety of algorithms used for the visualization and analysis. How it works is from the impurity data given to us from the segmentation, we take the data, and perform different types of operations to visualize the data. These operations consist of statistical distribution fitting, which fits the various probability distributions to data using the python library fitter, then they use Kernel Density Estimation (KDE) which uses the scikit-learn library, from that data given, they then do Maximum Likelihood Estimation (MLE) and Full width at half maximum (FWHM) to find the various peaks given from the KDE and then calculates the half maximum with the same data given from the KDE. Once these operations finish, we then use image scaling and aspect ratio preservation to scale and crop the image appropriately, following a distance calculation and scale calibration to load the image properly so the user can correctly see the data for statistical distributions.

Services Provided		
Service Name	Provided To	Description
Data Analysis	Interface	Works on statistical data to gain insight about what the data is doing
Plot Generation	Interface	Uses the data and the information gained from the data analysis to create and display plots to the user.

Services Required

Service Name	Provided From
Segmentation data	Segmentation Analysis

IV. Data Design

Our program uses a DataFrame from the pandas Python library, which functions as a flexible two-dimensional data structure similar to a spreadsheet, organizing information in rows and columns. This enables us to efficiently store and manage measurement data extracted from image segmentation processes, including attributes like size and shape. By structuring the data in this format, we can easily analyze and manipulate it. This is primarily useful for creating the visual representations such as histograms that depict the distribution of various measurement types. Beyond these core functions, the DataFrame provides a scalable foundation that can be adapted for additional analytical features in the future, depending on evolving client needs and design considerations. It is also easy to export to common file types such as excel or csv, which give the user greater flexibility over the way they represent data.

V. User Interface Design

Our team is refactoring an existing GUI design to better support the functionality we aim to add, as well as increase the general extendability of the UI for any potential changes in the future. Accessibility and intuitiveness are the main design goals for the new GUI application. See 'Figure 1', 'Figure 2', and 'Figure 3' in the appendix below for the existing GUI, our initial modified GUI, and the first version of our new GUI respectively. The existing GUI is bare, unintuitive, and presents challenges to new users about what the application does, can do, and order of operations. Our modified version of this UI simply adds buttons to extend functionality we included. The first version of the GUI is our ground-up approach to how our project team envisions the finalized application. We aim for a minimalist, stylistic structure that clearly defines the application process and potential functionality.

The following description is of our team's first version of the GUI, found in the appendix below in 'Figure 3'. The user interface will start by prompting the user to Load image/images. It will then go through a scaling and cropping cycle, before setting required parameters (Footprint type, Thresholding type, Blur type, Minimum particle area, size) for all images. Once these prerequisites have been met, a Run Segmentation button (or Run Batch Segmentation button) will begin the analysis. The segmented image and histogram will be displayed as output, as well as a defined section for plotted graphs using the collection of segmented data (for multi-image analysis).

The current application can be run by downloading/cloning the current github repository and running the 'main.py' file. Requirements are listed in the project README, with a 'requirements.txt' file for easily checking/installing all application requirements. We intend to

reconfigure the application into an executable download, which serves to further generalize the application and enhance the ease-of-use for researchers/application users.

VI. Glossary

SAM: Segment Anything Model

SOTA: State-of-the-art

GUI: Graphical user interface

CNN: Convolutional neural network

KDE: Kernel Density Estimation

MLE: Maximum Likelihood Estimation

FWHM: Full Width at Half Maximum

Image Segmentation: a technique that partitions a digital image into discrete groups of image segments.

Batch Segmentation: the process of dividing a large dataset of customer or user data into different segments based on specific criteria

VII. References

[1] Ronneberger, Olaf, Philipp Fischer, and Thomas Brox. "U-net: Convolutional networks for biomedical image segmentation." *Medical image computing and computer-assisted intervention–MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18*. Springer International Publishing, 2015

[2] "Introducing Meta Segment Anything Model 2 (SAM 2)" Meta.com. Accessed: Jan. 30, 2025. [Online.] Available: <https://ai.meta.com/sam2/>

[3] Ahmed Mahgoub Ahmed Talab, Zhangcan Huang, Fan Xi, Liu HaiMing, Detection crack in image using Otsu method and multiple filtering in image processing techniques, Optik, Volume 127, Issue 3, 2016, Pages 1030-1033, ISSN 0030-4026, <https://doi.org/10.1016/j.ijleo.2015.09.147>.
(<https://www.sciencedirect.com/science/article/pii/S0030402615012164>)

[4] S. Eddins. "The Watershed Transform: Strategies for Image Segmentation." Mathworks.com. Accessed: Jan. 30, 2025. [Online.] Available: <https://www.mathworks.com/company/technical-articles/the-watershed-transform-strategies-for-image-segmentation.html>

[5] "Comparison of Segmentation and Superpixel Algorithms" Scikit-image.org. Accessed: Jan. 30, 2025. [Online.] Available: https://scikit-image.org/docs/stable/auto_examples/segmentation/plot_segmentations.html

VIII. Appendices

Figure 1

← →

Footprint Type Edit

Thresholding Type Edit

Blur Type Edit

Min. Particle Area: Edit

Size ▾

Run Segmentation

Load Image Crop

Set Scale Segmented Image ▾

Figure 2

Footprint Type

Thresholding Type Edit

Blur Type Edit

Min. Particle Area: Edit

Size ▾

Run Segmentation

Run Batch Segmentation

Load Images Crop

Apply All Set Scale Segmented Image ▾

< >

Figure 3

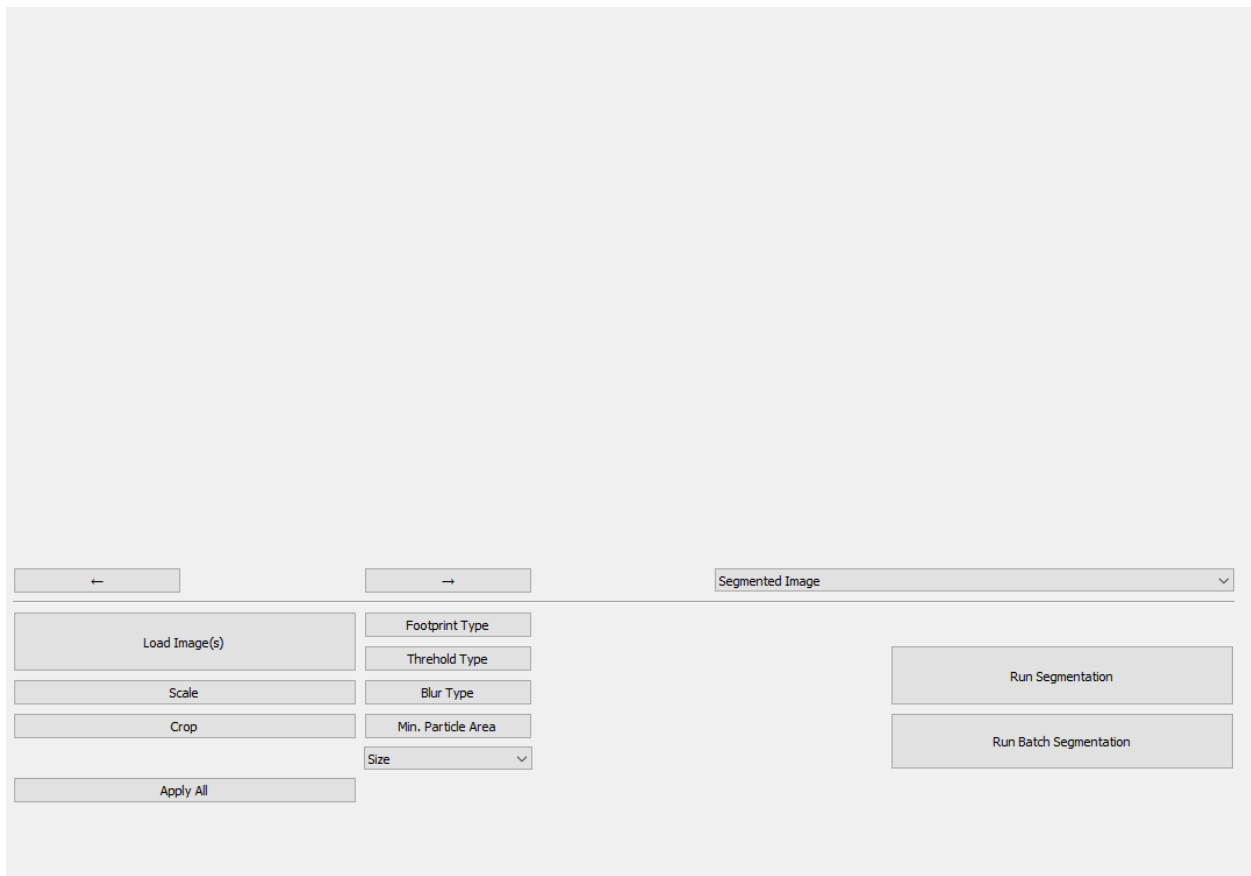


Diagram 1

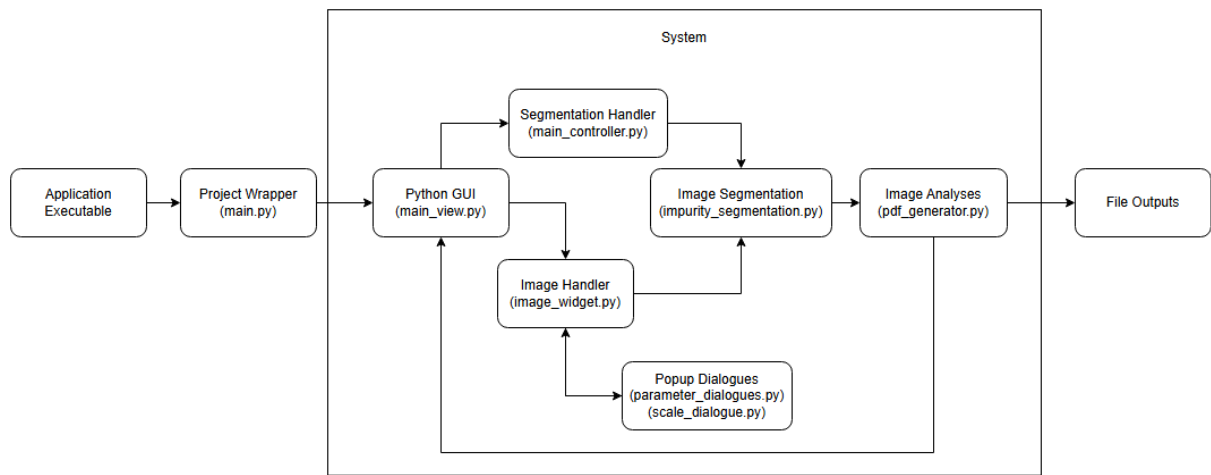


Diagram 2

